

1. Introducing .NET

The Microsoft .NET Framework is a collection of software technologies that began emerging from Microsoft Corporation around the earlier 2000s. According to Microsoft, **.NET** is a **Free, Cross-Platform, Open-Source** developer platform for building many different types of applications. With .NET, we can use multiple languages (C#, VB, F#, etc.), Editors (Visual Studio, Visual Studio Code, Visual Studio for Mac, OmniSharp, JetBrains Rider, etc), and Libraries to build for Web, Mobile, Desktop, Games, IoT, and more.

Historically speaking, when programmers used a Microsoft development language (e.g., VB6) or Microsoft programming framework (e.g., MFC, COM, or ATL), they had to resign themselves to building software that (by-and-large) executed only on the Windows family of operating systems. I.e. these applications were platform dependent. Using .NET, your applications can find happy homes on numerous operating systems, including Mac OS X, Solaris, AIX, and numerous flavors of Unix/Linux.

You can program for .NET in a variety of programming languages. However, any language you use for .NET programming must meet a set of minimum requirements in order to use the .NET class libraries. These requirements are known as the .NET Common Language Specification or **CLS**. Related to the CLS is the .NET Common Type System (**CTS**) which defines the basic data types (such as integer, floating point, and string) that .NET languages support. The CLS and CTS are in turn part of the Common Language Infrastructure (**CLI**). The CLI is an ISO standard and an ECMA standard.

The .NET Framework mainly provides two fundamental things:

BCL (Base Class Libraries)

CLR (Common Language Runtime)

1.1. What is BCL?

The **Base Class Library** provides a set of APIs and types for common functionality. It provides types for strings, dates, numbers, etc. The Class Library includes APIs for reading and writing files, connecting to databases, drawing, and more. Without BCL we can't write any code in .NET. So, BCL is also known as the basic building block of .NET Programs. BCL contains pre-defined classes and these classes are used for the purpose of application development. These classes are installed onto our machine when we install the .NET framework.

1.2. What is CLR?

CLR stands for Common Language Runtime and it is the core component under the .NET framework which is responsible for converting the MSIL (Microsoft Intermediate Language) code into native code.

When you compile one of your .NET programs, the program is generally compiled to a processor-independent intermediate language that resembles machine code. This intermediate language was once called Microsoft Intermediate Language (MSIL), and it's still often known by that name. Sometimes it's just called Intermediate Language (**IL**). But the most proper term is now the Common Intermediate Language (**CIL**).

When a .NET program is run on a particular machine, the CIL is compiled to the native code of the processor by a just-in-time (**JIT**) compiler. This two-stage compilation potentially allows for portability among various platforms and processors. The just-in-time compilation is performed by the .NET Common Language Runtime (CLR), which is part of the .NET system installed on end-user's machines.

The CLR manages the execution of .NET programs, and can prevent programs from causing damage to the user's machine. Thus, when you are programming for .NET you are said to be writing —**managed code**. One important aspect of managed code involves the management of memory. As object-oriented programming and class libraries have become more complex over recent years, common problems have arisen involving memory allocation. Very often it's not clear who is responsible for freeing a particular memory block. For that reason, the CLR implements garbage collection. The CLR can determine if a particular block of memory can no longer be referenced by a program, and then free such blocks of memory if required.

Microsoft makes available several languages to the .NET programmer. Which one you use is mostly a matter of personal taste. Some people program for .NET using Visual Basic .NET. Others use Managed C++, more formally known now as C++/CLI.

However, most .NET programmers have come to know and love C#, the programming language designed in conjunction with .NET. C# incorporates much of the basic expression and statement syntax of C, and has a rather cleaner object-oriented programming syntax than C++. One of the biggest difference being, C# does not require you to mess around with pointers. Traditional C-like pointers are supported in C# syntax, but they are normally relegated to interoperability with existing code. Rather than pointers, the .NET and C# programmer works with —references, and these references are usually implied rather than being syntactically explicit.

1.3. Different types of .NET Framework

The .NET framework is available in three different flavors

.NET Framework: .NET Framework is the original implementation of .NET. It supports running websites, services, desktop applications, and more on Windows OS Only.

.NET: .NET is a cross-platform implementation for running websites, services, and console applications on Windows, Linux, and macOS. .NET was previously called **.NET Core**.

Xamarin/Mono: Xamarin/Mono is a .NET implementation for running apps on all the major mobile operating systems, including iOS and Android.

Note: .NET Framework is Platform-Dependent while .NET or .NET Core is Platform Independent.

More on the .NET Framework here: <https://dotnet.microsoft.com/en-us/learn/dotnet/what-is-dotnet-framework>

1.4. Runtimes and SDKs

To run .NET programs on your machine, you'll need to have some software installed that is variously known as the .NET —runtime or —runtime components or —redistributable or —redistributable package. The term —redistributable means that a software developer like yourself can distribute the .NET runtime if it's part of an installation for an application that requires the .NET Framework to run.

You'll need the .NET runtime components to run .NET programs. To develop .NET programs on your machine, you'll also need to install the .NET Framework Software Development Kit (SDK). Both the runtime and the SDK are free and both are generally downloadable from the same or related Web pages.

.NET downloads here: <https://dotnet.microsoft.com/en-us/download>

Refresher Questions:

1. What is a namespace?
2. What is an assembly?
3. What is the difference between .NET Framework and .NET core?
4. How does .NET achieve platform independence?